

Microcode Paper — Draft Sections 3 & 4 (preview)

Contents

1	Microcode as an Implementational Layer	1
1.1	Microcode, Triads, and the Patch RAM	2
1.2	The Triad as an Issue Bundle and the Entry Path	4
1.3	Patch RAM Budget	5
1.4	Findings	6
2	Designing the Microcode Kernel	8
3	Implementation	9
3.1	State Layout and Residency	9
3.2	The Looped Round Body	10
3.3	Generation, Packing, and the Triad Budget	11
3.4	Verification	12
4	Evaluation	12
5	Curve25519 in Microcode	14
5.1	Representation and Layout	14
5.2	The Kernel: One Firing, Three Carry Chains	15
5.3	Installing Both Operations at Once	17
5.4	Triad Budget: Density, Not Capacity	17
5.5	Assembling X25519 Around the Patches	18
5.6	Performance	19

1 Microcode as an Implementational Layer

Prior work has studied microcode as an object to be reverse engineered and, latterly, as a surface on which to mount security mechanisms. We instead treat it as a programming target: a layer beneath the instruction set on which a cryptographic primitive can be expressed directly, and on which it can be made to run faster than any code the instruction set itself admits. This section introduces the layer in the terms a reader will need, namely what a triad is, what the writable patch RAM is, and by what mechanism a stock instruction is customised to run code of our choosing, and it then sets out the methodology that turns those ingredients into a verified patch on the Intel Goldmont Celeron N3350. The same methodology underlies every primitive in this work, namely the field arithmetic for P-256, Curve25519, secp256k1 and Poly1305 together with the Keccak- $f[1600]$ permutation of $??$, and we therefore present it once, in general terms, and instantiate it twice in the sections that follow.

Two properties of the layer motivate the entire effort, and both will recur as the source of every result we report. The first is that the micro-architectural integer register file is twice the size of the architectural one, comprising 16 general-purpose and 16 temporary registers, which lets a primitive keep far more state resident than the roughly 14 usable general-purpose registers of native x86 and thereby removes the register spills that dominate compiled code on a register-starved in-order core; this is the property that ?? exploits to hold an entire Keccak state in registers across all 24 rounds. The second is that a triad issues up to three micro-ops as a single sequential bundle, which yields a denser and more hazard-aware packing than a linear instruction stream affords, and in particular lets several carry chains advance at once; this is the property that Section 5 exploits to overlap the multiply–accumulate–carry chain of a reduced field multiplication. The price of admission is a hard budget of 128 triads, together with a body of hardware behaviour that is nowhere documented and that we were obliged to map empirically.

1.1 Microcode, Triads, and the Patch RAM

From macroinstructions to micro-ops. The x86 instruction set is a contract between software and the hardware that runs it, a stable interface that fixes what each instruction must appear to do while saying nothing about how the hardware is to achieve it, and it is not the language the core actually executes. A decode unit translates each architectural instruction, which we call a *macroinstruction*, into one or more internal *micro-ops* (μ ops) drawn from a simpler, RISC-like repertoire that the execution units actually run. Frequent and simple macroinstructions are handled by fast hardware decoders that emit a handful of μ ops directly, whereas complex or rare ones are instead decoded by a microcode sequencer. It is for these latter, microcoded instructions that the μ op program implementing the macroinstruction is held on an on-chip microcode ROM, the MSROM, from which the sequencer fetches it. Goldmont exposes more than two thousand distinct μ ops, and a single complex macroinstruction may expand to dozens of them. The encoding is *vertical*, meaning that each μ op resembles a compact RISC instruction with an operation field and a small number of operand fields rather than a wide horizontal control word. The reverse engineering of Koppe et al. [?] on AMD K8/K10 first established this structure publicly, organising the μ ops into operation classes for register arithmetic, loads, stores and special operations, and Borrello et al. [?] subsequently recovered the corresponding μ ops for Goldmont together with their operand and immediate encodings.

The triad. Inside the microcode store the μ ops are not laid out singly but grouped into *triads*. A triad is a bundle of three μ ops together with one *sequence word*, and it is the unit in which microcode is addressed, fetched, and issued: the sequencer advances triad by triad, presenting the three μ ops of a triad to the execution units before it moves on. The sequence word is the control-flow field of the triad. It selects what the sequencer does after the triad’s μ ops have issued, namely fall through to the next triad, branch to a named triad address, or signal that decoding of the macroinstruction is complete, and on Goldmont it can additionally impose a synchronisation barrier on the bundle, in the manner of an *lfence*. A triad is thus at once a packing unit and the granularity of control flow. It is a packing unit because its three μ ops are fetched and issued together as a single bundle, so that the quantity governing both the size of a program and the length of its dependency chains is the number of triads rather than the number of μ ops, and filling all three slots of every triad is in consequence the central concern of the packing described in Section 1.2. It is the granularity of control flow because the sequence word steers the sequencer one triad at a time and a branch can target only a triad boundary, never a μ op within a triad, which has the practical consequence, developed in Section 1.4, that an operation setting a condition and the branch that tests it must

be placed in the same triad. The MSROM on Goldmont holds on the order of eight thousand such triads, 7936 to be exact [?], which together form the μ op program that decodes the entire x86 instruction set. Against that backdrop the 128 triads of writable patch RAM introduced below amount to under two percent of the decode store, and it is within that small allowance that an entire cryptographic primitive must be expressed.

The encoding of a triad. The structure of a triad is worth making concrete, since it determines what a single μ op can express and therefore what the packer of Section 1.4 is working against. We follow the Goldmont encoding first documented by Ermolov, Goryachy and Sklyarov’s uCodeDisasm [?], whose μ op mnemonics our toolchain inherits by way of lib-micro. Each μ op is a 48-bit word. A 12-bit opcode names the operation, and for some operations also encodes a sub-mode such as a condition code or a data size; two 6-bit fields select the source registers and a third 6-bit field the destination, which for a memory operation instead supplies a second source. An operand need not be a register: when the top three bits of a source field are 001 that field’s remaining bits and the μ op’s two immediate fields together form a 16-bit immediate, and when they are 010 the field indexes a read-only constants ROM. A few mode bits and a two-bit parity check complete the word. The 16-bit ceiling on an immediate is exactly why a 64-bit constant must be built by composition, a point Section 1.4 returns to. The sequence word is a narrower 30-bit word, but its fields act on the triad rather than on data. Three 2-bit pointers bind its actions to particular slots; a four-bit control-flow field selects among returning to the caller, looping within the microcode, stopping, and signalling that decoding is complete; a branch-address field supplies a jump target that becomes conditional when the preceding μ op is a flag-setting test or subtract; and a three-bit synchronisation field can place a load fence or a full barrier on the bundle, which is the *lfence*-like behaviour noted above. These two fields together are the whole of control flow in the layer, and Section 1.4 records which of their forms behave reliably on this part.

The patch RAM. The MSROM is fixed at manufacture, but a CPU must be able to correct decode bugs after it ships. Vendors provide for this with a small writable microcode store, the MSRAM or *patch RAM*: in normal operation the firmware or operating system loads a cryptographically signed microcode patch into it at boot by writing the patch’s address to a dedicated model-specific register, and the store is volatile and must be reloaded after every reset. It is one of several microcode-sequencer arrays, the others holding the read-only μ ops, their sequence words, and the match table described below. On Goldmont the patch RAM occupies the region U7c00–U7e00 and holds at most 128 triads, and this ceiling, as Section 1.3 develops at length, is the binding resource of our entire approach. A patch is therefore a short array of triads, each consisting of three μ ops and a sequence word, written in exactly the format the MSROM itself uses.

Customising an instruction. A patch is inert until some macroinstruction is made to decode into it. This is the role of the *match-and-patch* mechanism, a table of 32 entries each of which redirects a read-only triad address to one in the patch RAM. When the sequencer is about to fetch the triad at a matched MSROM address it is instead sent to the corresponding triad in the MSRAM. Pointing a match entry at the MSROM entry point of a chosen macroinstruction therefore *hooks* that instruction, so that issuing it transfers control into our patch. Because each entry redirects an even-aligned pair of addresses at once, mapping both `src` and `src+1` to consecutive patch triads, only even addresses can be hooked. This is the mechanism, and the only mechanism, by which we install code of our own: we write triads into the patch RAM and aim a match entry at an instruction we are willing to repurpose.

Three lines of prior work make this practical on Goldmont, and our toolchain rests on all of them. The foundation is due to Ermolov, Goryachy and Sklyarov: beginning in 2020 they obtained Intel’s *Red Unlock* debug state on Goldmont through the Intel-SA-00086 Management Engine vulnerability, identified the undocumented `udbgrd` and `udbgwr` instructions that read and write the microcode arrays from software [?], and released *uCodeDisasm*, the disassembler that first documented the Goldmont μ op, triad and sequence-word encoding publicly [?]. Building on this, Borrello et al.’s CustomProcessingUnit (2023) reverse-engineered the patch and hook format, defined the semantics of some eight thousand μ ops, and provides a static decompiler together with an assembler from μ op mnemonics to triads [?]. We build most directly on Krog’s *lib-micro*, an eBPF-inspired C macro language for chaining μ ops into triads together with a C API that installs a patch and programs a match entry on an individual core, writing the sequencer arrays through the `udbgrd/udbgwr` interface rather than through the signed boot-time update path. Reaching that interface requires a part held in the Red-Unlock state above, which we reproduce on our Celeron N3350. Where this body of work used such access to study or to harden the CPU, we use it to make a cryptographic kernel the body of a hooked instruction, and the rest of the paper is concerned with doing so well. The two ways in which a primitive stresses the mechanism, control flow on the one hand and arithmetic density on the other, are the subjects of ?? and Section 5 respectively.

1.2 The Triad as an Issue Bundle and the Entry Path

The entry path. Every primitive shares a single entry path that instantiates the customisation mechanism of Section 1.1. We hook an otherwise unused macroinstruction, the `vmwrite` whose opcode is `0x0cd8`, by programming a match register so that its MSROM entry point is redirected into our patch in the MSRAM. The native side places the operand pointers in fixed architectural registers, with a base pointer in `RCX`, and issues a single `vmwrite`, whereupon control transfers into the patch. The patch reads its inputs, computes entirely in registers, writes its outputs through the base pointer, and returns when its final triad carries the sequence word that signals decode-complete. Because the hook is placed on a single static instruction, a complete operation executes within one firing, which, as Section 1.3 explains, is the property that makes register residency worthwhile. When two patches must be resident simultaneously we hook a distinct macroinstruction for each, since a single hooked instruction can be redirected to only one entry point; the Curve25519 field operations of Section 5, for example, fire the multiplication on `vmwrite` and the squaring on `vmread`, whose opcode is `0x0618`.

The triad as an issue bundle. Where Section 1.1 described the triad as the microcode store sees it, we now describe how we use it. We program each triad as a bundle of up to three μ ops together with its sequence word, and we rely throughout on a property that we verified rather than assumed, as reported in Section 1.4: the three slots execute with full sequential semantics over the register file. A value written in an earlier slot is visible to later slots, which is read-after-write forwarding; a read in an earlier slot observes the old value of a register that a later slot overwrites, which is the write-after-read case; and when two slots write the same register the later slot prevails, which is the write-after-write case. This holds identically for general-purpose and temporary registers, and for condition flags as well as for data. In practice the triad behaves like a small statically scheduled VLIW word with intra-bundle forwarding, so that a short dependent chain can be folded into a single issue group rather than spread across three instructions while independent work fills the remaining slots. The layer is most expressive precisely where compiled native code is weakest, namely in the dependent reductions and carry chains that the instruction set architecture would

serialise; this is the lever behind both worked examples, where ?? folds a balanced parity tree into successive slots and Section 5 fuses a multiplication, an accumulation and a carry capture into one bundle.

Packing constraints. Two rules bound what may share a triad, and both are enforced by the hardware on pain of a hang or a silently incorrect result. The first is that at most one memory μop may occupy a triad, because a second load or store in the same triad hangs the core, which has only a single first-level-cache data port. The second is that a value produced by a load in an earlier slot is not forwarded to a later slot of the same triad, unlike a register result, so that a consumer of a freshly loaded value must begin a new triad. The asymmetry is worth stating explicitly, since arithmetic-to-arithmetic forwarding within a triad is permitted whereas load-to-arithmetic forwarding is not, and it is this boundary condition that most often terminates a triad.

A self-verifying generator. Because a single mis-scheduled μop produces a silently incorrect result and every hardware trial risks hanging the machine, we do not author triads by hand where we can avoid it. For each primitive a generator emits an ordered op-list over real register names, simulates that op-list against a model of the register file and memory that incorporates the addressing constraints of Section 1.4, and asserts the result against a reference implementation over many random inputs and against published known-answer vectors. Only thereafter does a packer lower the op-list to triads and the generator emit the C array. The packer is a single greedy left-to-right pass that closes the current triad whenever it fills its three slots, whenever a second memory operation would enter it, or whenever an operation would consume a register loaded earlier in the same triad, so that it mechanically enforces the two packing rules above, and it isolates control-transfer triads where required by Section 1.3. This discipline moves all correctness debugging into software, where it is immediate and safe, and converts the hardware-specific scheduling rules into a property of the toolchain rather than a burden on the programmer. The same procedure of generation, simulation, verification and packing underlies the Keccak permutation of ??; the small, densely irregular field kernels of Section 5 are instead hand-scheduled, but held to the identical verification discipline.

1.3 Patch RAM Budget

The writable patch RAM on Goldmont holds 128 triads, occupying the region U7c00–U7dfc in the MSRAM, in which each triad occupies four address units and must begin on an even address, and within which a small sub-range is reserved as a staging area and is not available to a primitive. This ceiling of 128 triads is the binding resource of the entire approach, and it shapes the design at the level of the algorithm rather than merely at the level of the code.

The budget selects the strategy. Because a patch is small, the central question for each primitive is how to accommodate a complete operation, including any iteration it involves, within 128 triads while keeping its working set resident. The answer is primitive-specific but is in every case a budgetary decision. An operation whose natural straight-line form would overflow the cap is restructured so that a single resident body is reused: an iterated computation is looped in place by means of a backward branch in the sequence word, with loop counters and constant tables held in the operand buffer rather than encoded in the program, so that the triad count becomes independent of the iteration count. This is the route taken by Keccak in ??, whose 24 rounds are looped over one resident round body. Where looping is too costly relative to the work performed per iteration, we instead fold a fixed and small number of iterations into one firing, as in the

processing of two reduction words per `vmwrite` in the word-by-word Montgomery primitives, and re-enter from the native side, thereby trading a little additional input and output for remaining within the cap. By contrast, a primitive whose straight-line form already fits, as the Curve25519 field operations of Section 5 do at 42 and 66 triads, is bound not by the cap but by how densely its arithmetic can be packed. In every case it is the relation between the work and the cap, rather than the instruction mix, that fixes the macro-structure of the implementation.

Spend the budget on residency, not on movement. Temporary registers do not survive across firings, so any value that must outlive a hooked instruction has to be carried in an architectural register or spilled to memory. This makes a single firing per operation the default arrangement, in which the inputs are loaded once in a prologue, the operation is computed entirely in registers, and the outputs are stored once in an epilogue, so that the memory traffic is paid once rather than once per iteration. The doubled register file is what renders this feasible, and we regard every triad spent on address arithmetic or data movement as a triad taken from the primitive itself. A concrete illustration arises in our own work, in which a variant that rebuilt an addressing base on each iteration in order to free one register measured strictly slower than a variant that kept the base pinned and instead borrowed a normally reserved register as scratch, because the cost of the rebuild exceeded the cost of the spill it removed. Under a fixed triad budget, the minimisation of data movement is the optimisation that matters most.

1.4 Findings

Using microcode as a target required us first to establish the undocumented behaviour of this particular Goldmont part, namely which μ ops exist and what they compute, how operands and flags are encoded, and which combinations are illegal. Each finding reported below was isolated by a dedicated probe and is now encoded as a rule in the generator and packer, so that a software-verified op-list lowers to a patch that runs correctly on its first hardware trial. We group the findings under the headings of the μ op repertoire, the execution and flag semantics, the addressing, the control flow, and the performance.

Micro-op repertoire. The vertically encoded μ ops expose a RISC-like set that is sufficient for field arithmetic, comprising 64-bit `ADD`, `XOR`, `SHL` and `SHR`, the rotation `ROL`, which is a correct single μ op for every rotation amount, the bitwise operation `NOTAND`(d, a, b) = $(\neg a) \wedge b$, register moves by means of `ZEROEXT`, and a $64 \times 64 \rightarrow 128$ multiplication `MUL_DSZ64_DRR`(hi, a, b) that preserves a , returns the low half in b , and places the high half in hi . Immediate operand fields are 16 bits wide, so a 64-bit constant must be materialised by composition, using `ZEROEXT` followed by `SHL` and `XOR`, rather than in a single operation. Two pitfalls cost us considerable time. The first is that the memory operand forms, such as `ZEROEXT_DSZ64_DM`, do not perform general addressing but instead read a field of the triggering macroinstruction, so that general memory access is available only through the dedicated load and store operations. The second is that operations with an implicit memory operand cannot be used inside a patch at all. We therefore restrict the generator to register-only arithmetic together with the explicit load and store operations.

Execution and flag semantics. Intra-triad register dataflow is fully sequential, as described in Section 1.2, with the single exception that the result of a load does not forward within a triad; store-to-load forwarding across triads within a patch does function, however, which is what makes the spilling and reloading of scratch values safe. A 64-bit multiplication may occupy any slot of a triad, an earlier belief in a slot-zero restriction having not survived testing. The flags are the subtlest

aspect of the machine, and we found two distinct flag domains. The first is a per-temporary-register condition that is set by an `ADD` and consumed by a conditional `SETCC`, and that persists until the next `ADD` to that register. The second is the architectural `RFLAGS`, which is frozen at patch entry and which is what `ADC` and the conditional branch read. The bridge from the first domain to the second is unreliable in large patches. Three practical consequences follow and are encoded in the generator. All multi-word carry chains are built from per-register `ADD` and `SETCC` pairs rather than from `ADC`; `SETCC` writes correctly only to a temporary register and never to an architectural one, so a carry must be materialised in a temporary and then moved out; and a temporary needed across iterations must never be clobbered by an intervening carry computation. The first of these is precisely what enables the parallel carry chains of Section 5, and its boundary is precisely what limits them on the saturated representation discussed there.

Addressing. Loads and stores function only with a specific data-segment selector on this part, and a selector copied from unrelated code hung the core unrecoverably, so the working value is machine-specific and is fixed once. The immediate displacement is an 8-bit signed byte offset spanning ± 128 bytes, and an early layout that placed scratch beyond that range wrapped silently and corrupted state, a fault at first misdiagnosed as a dependency error. We therefore centre the operand buffer beneath its base pointer so that all displacements remain in range, and we use the register-indexed addressing form, which carries no such limit, for the tables and counters that lie beyond the reach of the immediate. As noted above, at most one memory operation may occupy a triad.

Control flow. A backward conditional branch encoded in the sequence word transfers control to an arbitrary triad and reads the architectural flags, ignoring any register operand encoded alongside it, and the arithmetic operation that sets the tested flag must occupy the same triad as the branch. This is the mechanism that underlies the in-place looping of Section 1.3, and `??` depends on it directly. Because temporary registers do not persist across firings, any state that must survive between firings has to reside in an architectural register or in memory, which is precisely why a complete, iterated operation is kept within a single firing wherever the triad budget permits.

Performance. The layer has two regimes of per-triad cost, and conflating them is the easiest way to misjudge a projected result. Measured as throughput, across many independent firings that the out-of-order front end is able to overlap, a triad costs approximately 0.49 cycles. A single dependent operation, by contrast, pays the latency, and in the absence of any overlap between firings and in the presence of a per-iteration branch the effective cost rises to roughly one triad per cycle. A primitive is therefore fast in this layer to the extent that its triad count is low and its critical path short, rather than because individual operations are cheap. The improvements we report across the primitives, which match or exceed both GCC at `-O3` and a state-of-the-art superoptimiser on most operations, derive from this structural source: the doubled register file removes the spills that dominate compiled code on an in-order core, and the three-wide hazard-aware packing shortens dependent chains. The advantage is structural, deriving from the elimination of data movement that the instruction set architecture cannot avoid, rather than from a higher peak throughput; and because the 128-triad cap forecloses extensive unrolling, the margin is modest on kernels that are already tight and latency-bound and large on kernels that are spill-heavy. The two worked examples that follow are chosen to exhibit both ends of this range.

2 Designing the Microcode Kernel

The first worked example instantiates the methodology of Section 1 on the Keccak- $f[1600]$ permutation. Keccak is the most demanding test of the layer in this work: in contrast to the straight-line field-arithmetic primitives it is a 24-round loop with a per-round constant table, and it therefore exercises in-place looping, indexed addressing, and conditional control flow in addition to dense arithmetic packing. It is also the example in which the structural advantage of microcode is most readily stated, and, as we shall see, the example in which the limits of that advantage are most clearly exposed. This section sets out the design decisions; the concrete mechanics that realise them are deferred to Section 3.

The structural opportunity: residency across all rounds. A Keccak state consists of 25 lanes of 64 bits. Native x86-64 exposes approximately 14 usable general-purpose registers, with the consequence that hand-written assembly cannot hold the entire state in registers; it spills to the first-level cache continuously, and the fastest scalar implementations recover residency only by fully unrolling all 24 rounds. The micro-architectural register file is 32 entries wide, comprising 16 general-purpose and 16 temporary registers, which is sufficient to hold the complete 25-lane state resident not merely within a single round but across all 24. Our first and governing design decision is to rest the entire kernel on this property: the 25 lanes are loaded once, the 24 rounds are evaluated with the whole state resident in registers, and the lanes are stored once at the end. Residency is obtained without unrolling, which the 128-triad capacity of Section 1.3 would not in any case permit, and the resulting register headroom is then spent on per-round transformations that register-starved native code is unable to perform.

One firing for the whole permutation. The second decision concerns where the round boundary lives. Rather than fire the patch once per round, which would force the state out to memory and back at every boundary and pay the hook cost 24 times, we evaluate the entire permutation within a *single* hooked `vmwrite` firing and loop the round body internally by means of a backward branch. Because control never leaves the patch, the temporary registers persist for the lifetime of the permutation, and the state therefore stays resident across all 24 rounds with no boundary traffic. Only a prologue and an epilogue, executed once each, move the state between registers and memory. This choice is what makes the residency of the previous paragraph realisable in practice.

Why a single looped body is correct: the structure of π . The natural objection to looping a Keccak round is that π relabels lane positions, so that the lane held in a given register would appear to drift from one round to the next, which would seem to demand either per-round register renaming or two alternating body copies. The objection is unfounded, and the reason is structural. The permutation π on the 25 lanes consists of one fixed point, namely lane 0, together with a single 24-cycle. If π is applied *in place* along that cycle — that is, by walking the cycle and moving each element into its destination register while folding in the exclusive-or of θ and the rotation of ρ — then every lane is returned to its canonical register by the end of the round. Round $r+1$ consequently observes exactly the register layout that round r observed, so that a single identical body may be looped 24 times with no renaming and no two-body alternation scheme. This observation is the linchpin of the design: it is what allows the static footprint to be one round body rather than 24, and hence what allows residency and the 128-triad budget to coexist.

Borrowing a 32nd register. Holding the 25 state lanes, the five mixing lanes D of θ , the buffer base, and a working temporary simultaneously requires one register more than the architecturally nameable file affords once the base pointer is reserved. Our third decision is therefore to borrow `RSP` as a 32nd data register, saving it to the buffer in the prologue and restoring it in the epilogue. This is safe because, as established in Section 1.4, the patch never touches the stack. The substitution is not cosmetic: it is precisely what permits the five D values to remain resident in registers throughout a round instead of being re-read from memory once per lane, which would otherwise cost 25 additional memory triads per round.

Spending the headroom. The final design decisions concern how the freed registers are used, each being a transformation that native code cannot make because it lacks the spare registers to do so. The state is kept fully resident, so that input and output are paid once per permutation rather than once per round. The five D values are kept resident, as just described. And χ , which would otherwise force two auxiliary moves per row to preserve values it is about to overwrite, is restructured to use the now-dead D registers as scratch and so requires no auxiliary moves at all. To these we add two latency-shortening choices on the round’s critical path: a balanced parity tree for θ , and a loop counter that stores the round constant’s byte offset directly so that the constant fetch needs no scaling arithmetic. The mechanics of all five appear in Section 3, and their measured contributions in Section 4. One further decision is forced rather than chosen: the loop counter is advanced *upward*, because the decrement-style operations that a count-down would require do not behave correctly on this part, as recorded in Section 1.4.

3 Implementation

We now give the concrete realisation of the design of Section 2: the register and buffer layout, the triad-level form of each round stage, the generator and packer that produce the patch, and the verification that accompanies it.

3.1 State Layout and Residency

Register allocation. The 25 lanes, indexed by $i = 5y + x$, are assigned to a fixed set of 25 register names, namely the 13 general-purpose registers `RDI`, `RSI`, `RBX`, `RDY`, `RBP`, `R8`...`R15` followed by the 12 temporaries `TMP0`...`TMP11`. The register `RCX` holds the buffer base and is never repurposed, while the remaining five registers, namely `RAX` and `TMP12`...`TMP15`, serve as scratch: they hold the five resident D values of θ during a round and the temporaries of χ once those values are no longer needed. With `RSP` borrowed as the 32nd data register (Section 2) the file is exactly filled, `RSP` serving as the working register for the parity tree, the ρ rotation, and the π cycle walk.

Buffer organisation. The operation is backed by a single contiguous array of 56 lanes, with the base pointer centered at `RCX = &buf[16]` so that every displacement fits within the 8-bit signed offset field discussed in Section 1.4, as summarised in Table 1. Only the 25 state lanes are accessed by the prologue and epilogue. The column-parity scratch is spilled and re-read within a round, whereas the round-constant table lies beyond the reach of the immediate offset and is therefore addressed through the register-indexed load form. The loop counter is stored not as a round number but as the byte offset of the current constant, so that the constant fetch requires no scaling arithmetic on its critical path.

Table 1: Operand buffer of 56 lanes, with the base register centered at `RCX=&buf[16]`.

lane	offset(RCX)	name	contents
0–24	−128...+64	state	resident in 13 GPR + 12 TMP during a round
25–29	+72...+104	DSCR	θ -C parity scratch (spilled, re-read)
30	+112	COUNTER	RC byte-index, initialised to 128 and advanced by 8 each round
31	+120	RSPSAVE	saved RSP (borrowed as a data register)
32–55	(indexed)	RC table	24 round constants, addressed only by register index

3.2 The Looped Round Body

The permutation is the prologue, a body looped 24 times by a backward branch, and the epilogue. The prologue loads the 25 lanes using one single-load triad per lane, since at most one memory operation may share a triad (Section 3.3); the epilogue stores them symmetrically. We describe the body in order of execution.

θ and the residency of D . The column parity $C_x = \bigoplus_y A[x+5y]$ stands at the head of every round’s dependency chain, so we shorten its latency. In place of the reference implementation’s depth-four left-associated chain we emit a balanced tree of depth three, using RSP as a second accumulator so that the two halves are computed in parallel, and the packer places the dependent exclusive-or operations in successive slots of a single triad, as shown in Listing 1. The five values C_x are then spilled to the buffer, because the registers are required immediately afterwards, and the mixing lanes $D_x = C_{x-1} \oplus \text{rol}(C_{x+1}, 1)$ are computed into the five scratch registers and kept resident for the remainder of the round, eliminating the 25 per-lane reloads of D that earlier versions of the kernel incurred.

Listing 1: The depth-three parity tree of θ , in which a single column’s tree is packed into the slots of one triad while the spill of the previous column is interleaved.

{ XOR(RAX,RDI,R8),	XOR(RSP,R13,TMP2),	XOR(RAX,RAX,RSP),	NOP_SEQW }
{ XOR(RAX,RAX,TMP7),	STAD(RAX,RCX,72),	XOR(RAX,RSI,R9),	NOP_SEQW }

The fused θ -apply, ρ , and π stage. By the argument of Section 2, π is one fixed point plus a single 24-cycle, so applying it in place along that cycle returns every lane to its canonical register. Concretely, we walk the cycle and move each element into its destination register while folding in the exclusive-or of θ and the rotation of ρ , reading D directly from registers with no memory access. The fusion costs one exclusive-or and one rotation per lane, and RSP holds the one element displaced at the head of the cycle.

The χ stage without auxiliary moves. The step χ computes, for each row, $A_x \oplus = (\neg A_{x+1}) \wedge A_{x+2}$, which corresponds exactly to the NOTAND primitive. Performed naively in place, the overwriting of A_x would destroy a value the subsequent lanes still require, which forces two auxiliary moves per row, or ten per round, in register-starved native code. Because the five D registers are now dead, we have the headroom to compute all five NOTAND terms of a row first, placing them in scratch, and then to apply the five in-place exclusive-or operations, as shown in Listing 2, so that no auxiliary moves are required at all. This is the single largest measured improvement (Section 4), and it is a direct consequence of residency.

Listing 2: The move-free χ stage, in which the five NOTAND terms of a row are computed into the now-dead D -registers before the five in-place exclusive-or operations are applied.

```
{ NOTAND(RAX,RSI,RBX), NOTAND(TMP12,RBX,RDX), NOTAND(TMP13,RDX,RBP), NOP_SEQW }
{ NOTAND(TMP14,RBP,RDI),NOTAND(TMP15,RDI,RSI),XOR(RDI,RDI,RAX),      NOP_SEQW }
{ XOR(RSI,RSI,TMP12),   XOR(RBX,RBX,TMP13),   XOR(RDX,RDX,TMP14),   NOP_SEQW }
```

ι and loop control. Because the counter lane stores the byte offset of the current round constant directly, ι reduces to a load of the index, an indexed load of the constant, and a single exclusive-or into lane 0, with no scaling arithmetic on the path from ι to lane 0. The counter is then advanced by 8 and persisted in the spare slots of the same triad, after which a single forced triad performs the loop test and the backward branch. In that triad an exclusive-or of the index against the end marker sets the architectural zero flag, and a conditional branch within the same triad returns control to the top of the body for as long as the index differs from 320, which lies just past the last constant, thereby producing exactly 24 iterations, as shown in Listing 3.

Listing 3: Loop control, in which the flag-setting exclusive-or and the backward conditional branch must occupy the same triad.

```
{ XOR(TMP12,TMP14,320), NOP, UJMPCC_CONDNZ(TMP12, 0x7c68 /*loop_top*/), NOP_SEQW }
```

3.3 Generation, Packing, and the Triad Budget

No hand-written triads. In accordance with Section 1.2, no triad is written by hand. The generator emits the round as an ordered op-list over real register names, and a single greedy left-to-right packer lowers that list to triads, closing the current triad whenever it would exceed three slots, whenever a second memory operation would enter it, whenever an operation would consume a register loaded earlier in the same triad, and around the forced loop-test triad. The packer therefore enforces the hardware scheduling rules mechanically rather than leaving them to the programmer.

The one-memory-operation rule. The packer admits at most one memory operation per triad, which is why the prologue and epilogue are 25 single-lane triads each. A targeted probe refined the cause of this rule. A single triad bearing two memory operations executes correctly in every combination of loads and stores we tested in isolation; however, a kernel in which many such triads are packed consecutively — as arises naturally when the 25-lane prologue and epilogue are compressed two lanes to a triad — faults the machine. The binding constraint is thus the sustained throughput of the memory cluster rather than the legality of an individual triad, and we retain the one-operation-per-triad rule in production.

Budget. The complete permutation compiles to 108 triads, which lies comfortably within the 128-triad patch RAM. These comprise a 26-triad prologue, consisting of one `RSP` save and 25 single-lane loads, a 56-triad body that includes the constant lookup and the loop test, and a 26-triad epilogue, consisting of 25 stores and the `RSP` restore. The body begins at `U7c68`, which is the branch target. Because the body is looped rather than unrolled, the static footprint is independent of the number of rounds, whereas the dynamic count — the number of triads actually executed per permutation — is $26 + 24 \times 56 + 26 = 1396$. The 20 triads of headroom beneath the cap are genuine slack, but, as Section 4 explains, they cannot be spent on unrolling, because even a single additional round body would not fit alongside the existing one.

Table 2: Best result for each contender over 24 compiler-and-optimisation configurations, measured base-pinned and same-process, where a smaller value is better.

contender	min (cyc/perm)	median	best config
microcode (looped)	1910	1916	gcc-13 -Os
opt641cu24	2045	2054	gcc-11 -Os
x86_64_asm	2061	2069	clang-14 -O2
opt641cu24shld	10053	10064	clang-17 -Os
x86_64_shld	10086	10097	gcc-13 -O2

3.4 Verification

Before any triads are emitted, the generator executes the op-list on a register-and-memory simulator that models the 8-bit signed-offset limit, so that an out-of-range displacement manifests as an assertion failure in software rather than as a silent corruption of state on hardware, and asserts equality with a reference Keccak over 64 random states across all 24 rounds. On hardware the installed patch is then checked in two ways: first lane by lane against the same reference, and second, more stringently, against the published $f(0)$ and $f(f(0))$ known-answer vectors of the Keccak team, so that a correlated error present in both the reference and the patch cannot escape detection. The all-zero anchor reproduces the canonical value `lane0 = 0xF1258F7940E1DDE7`.

4 Evaluation

Measurement methodology. Timing on this part is subject to a pitfall that invalidates naive comparisons. The `rdtsc` instruction counts at a fixed reference rate of approximately 1.1 GHz, whereas the core bursts to approximately 2.4 GHz under load, with the consequence that the same work is reported as roughly 2.2 times more or fewer ticks depending on the power state during measurement. Comparing a fresh measurement against a baseline captured in a separate run is therefore meaningless. We instead time the microcode and all of the scalar contenders back to back within a single process, after a warm-up phase has driven the core to a steady frequency, so that both observe the same clock and the ratio between them is frequency-invariant; for absolute cycle counts we additionally pin the core to its base frequency so that the reported ticks approximate true cycles. We compare against the genuinely fastest scalar baselines from SUPERCOP, swept over 24 compiler-and-optimisation configurations with the best result selected for each contender, in the same manner as the SUPERCOP autotuner.

Result. Under base-frequency, same-process measurement the looped permutation completes in 1910 cycles per permutation, as against 2045 cycles for the fastest scalar contender, `opt641cu24`, and 2061 cycles for the hand-tuned `x86_64_asm`, as reported in Table 2. This corresponds to a ratio of 0.934, an improvement of approximately 7%, and the microcode is the fastest of all contenders. The ratio is the robust quantity, since the same harness measured at burst frequency reports approximately 874 cycles against 942, the identical ratio of 0.93 scaled by the 2.4/1.1 power-state factor.

The source and the bound of the improvement. The margin is modest by design, and the latency model of Section 1.4 explains why. A single isolated firing sustains 0.49 cycles per triad, but this is a throughput figure obtained by overlapping independent firings, whereas the

Table 3: The improvement assembled from successive structural optimisations, expressed as the ratio to the naive residency-only baseline.

step	change	ratio
0	per-lane D in buffer, with a naive χ using auxiliary moves	1.00×
1	D resident in registers via the borrowed RSP , with no base rebuild	0.99×
2	move-free χ using the dead D -registers as scratch	0.95×
3	depth-three θ tree and byte-indexed RC , shortening the critical paths	0.93×

looped permutation is a strict dependency chain, in which round N requires round $N-1$, with a backward branch on each round and no overlap between firings, so that it pays the true latency of approximately 1.37 cycles per triad and the 1396 dynamic triads accordingly cost approximately 1910 cycles. The microcode is therefore latency-bound at roughly one triad per cycle, whereas the hand-tuned scalar assembly extracts approximately three operations per cycle from the rotate and **ANDN** instruction-level parallelism of Goldmont. We cannot close this gap with additional parallelism, because the chain $\theta \rightarrow \rho \rightarrow \pi \rightarrow \chi$ is inherently sequential and the 128-triad cap precludes the unrolling that would expose independent work. The entire advantage is consequently the per-round work that native code cannot perform: full register residency, with the state input and output paid once per permutation rather than once per round; the borrowed 32nd register that keeps D resident; and the move-free χ stage. Table 3 shows the improvement assembled from these structural contributions, each measured back to back.

In summary, a complete and known-answer-verified 24-round Keccak- $f[1600]$ runs entirely in Goldmont microcode, looped within a single **vmwrite** firing, in 108 triads, and outperforms the fastest hand-tuned scalar assembly on this microarchitecture by approximately 7%. The result is a clear demonstration of the character of the layer established in Section 1: its advantage is structural, deriving from the elimination of the spills and data movement that the instruction set architecture cannot avoid, rather than from a higher peak throughput, and it is greatest precisely where native code is most constrained by the shortage of registers.

5 Curve25519 in Microcode

The second worked example carries the methodology of Section 1 from a permutation onto field arithmetic. The Keccak study of ?? concerned a single large kernel, evaluated once per message block, whose difficulty was that its state did not fit in the architectural register file. A Curve25519 field operation presents the opposite profile in every dimension that the microcode layer is sensitive to. It is small, it is straight-line, it is dominated by integer multiplication, its operands occupy only a handful of registers, and a single scalar multiplication invokes it on the order of a thousand times. The object we place in the patch RAM is therefore not the scalar multiplication but the two field operations beneath it, the multiplication and the squaring modulo the prime $p = 2^{255} - 19$, and the experimental question is whether a hand-scheduled microcode encoding of those two operations can be made faster than the best native code that exists for them: a modern optimising compiler, a formally verified arithmetic generator, a dedicated cryptographic superoptimiser, and the hand-written assembly of the primitive’s own designers.

Because the field element fits comfortably in registers, the lever that drove the Keccak result — whole-state residency — is unavailable and unnecessary here. The lever instead is the structure of carry propagation, and the remainder of this section is organised around it: Section 5.1 establishes why the carry chain, rather than the multiplication, is what binds a native implementation; Section 5.2 describes the triad kernel that breaks that bind by maintaining three carry chains at once; Section 5.3 and Section 5.4 treat the installation of the two patches together and the packing pressure peculiar to this primitive; Section 5.5 describes how the inlined scalar multiplication is built around the two patches; and Section 5.6 reports the measurements and states plainly where the advantage ends.

5.1 Representation and Layout

A field element is represented in the unsaturated radix-2⁵¹ form: five 64-bit words, each holding 51 significant bits and leaving the upper 13 bits free. This is the representation that fiat-crypto emits and that the Bernstein–Schwabe `amd64-51` code uses, and adopting it unchanged is what later permits the microcode field operations to be dropped into those native frameworks without disturbing anything around them. A multiplication is the 5×5 schoolbook of twenty-five partial products, and the reduction implied by $2^{255} \equiv 19 \pmod{p}$ is folded into the cross terms so that the wrapped products simply enter the lower output limbs scaled by 19, with no separate reduction pass:

$$c_i = \sum_{j \leq i} a_{i-j} b_j + 19 \sum_{j > i} a_{5+i-j} b_j, \quad 0 \leq i < 5.$$

Squaring is encoded as a separate patch rather than reusing the multiplication, because the symmetry $a_i a_j = a_j a_i$ collapses the twenty-five products to fifteen, with the off-diagonal terms taken with a coefficient of two. The coefficients this introduces — the doublings $2a_i$ and the reduction folds $19a_i$ — are not recomputed inside the patch. They are produced once by the caller using address-generation (`lea`) and integer-multiply (`imul`) instructions on the native side and handed to the patch in registers, so the kernel receives operands already shaped for its partial products and spends none of its triads forming them.

Register layout. A field operation is carried out entirely in registers. The caller loads the five limbs of each operand into a fixed set of general-purpose registers immediately before the triggering instruction, and reads the five result limbs from a fixed set of registers afterwards, so a field element is never spilled to memory while the operation runs. The two operations differ only in what the caller

must prepare in advance. For squaring, the four doublings $2a_0, \dots, 2a_3$ and the two folds $19a_3$ and $19a_4$ are formed on the native side with address-generation and integer-multiply instructions and delivered alongside the five limbs, and the result returns in `RDI, R9, R10, RBX, RAX`. For multiplication the caller delivers the second operand unmodified, and a three-triad preparation stage at the head of the patch forms the folds $19b_1, \dots, 19b_4$ in place while parking the original limbs in temporary registers for the cross terms that still require them, the result returning in `R15, R13, R9, R10, RAX`. Throughout the kernel a small, fixed set of temporary registers holds the running state of the accumulation: one register accumulates the low halves of the partial products (`TMP0`), a second accumulates their high halves (`R8`), a third sums the carries that the low-half additions produce (`TMP9`), and two further registers hold the low and high pieces of the carry passed from one limb to the next (`TMP8` and `TMP1`). The kernel of Section 5.2 is written against exactly this layout, and it is this register-resident discipline that later permits successive field operations to be chained to one another without a return to memory, as Section 5.5 describes.

A consequence of leaving thirteen bits free in every limb is that the carries produced while the partial products are summed are short and mutually independent. Because each limb is reduced to 51 bits, the value carried into the next limb occupies at most the complementary 13 bits, and the carry out of one limb does not depend on a carry buried inside another. This is a property of the layout itself, independent of any particular implementation, and the kernel of Section 5.2 is built around exploiting it.

Where the advantage comes from, and where it stops. It is worth stating precisely what this representation buys the microcode, because the point is easy to overstate. Within the 5×51 representation the binding resource for native code is carry propagation, not multiplication. The x86-64 architecture exposes a single carry flag, so a native accumulation must serialise its carries through the add-with-carry instruction, each instance of which both reads and writes that one flag, and the two-arithmetic-unit in-order target core cannot overlap that dependent flag chain with the independent multiplications that feed it. The microcode layer’s advantage lies exactly here: it advances several carry chains at once, each on its own private flag, in the manner that Section 5.2 describes, and this is what makes it the fastest 5×51 field backend we measured (Section 5.6). It is not, however, a claim that carry propagation is the bottleneck of X25519 in general, and a single observation makes the limit plain: the fastest implementation on this core, `amd64-64`, is itself built on long add-with-carry chains and still wins. It does so for two reasons that have nothing to do with carry handling. First, its saturated 4×64 algorithm performs fewer multiplications — sixteen partial products against our twenty-five — on a core whose multiplier, not its carry logic, is the scarce resource. Second, being ordinary native code it pays none of the per-firing dispatch overhead that our hooked operations incur on the order of two and a half thousand times per scalar multiplication. The honest measure of the layer’s advantage is therefore the comparison in which only the field operations change: with the microcode substituted into the `amd64-51` framework, leaving the ladder, inversion, and swap untouched, it is about eight percent faster end to end, and the squaring and multiplication on their own are roughly twenty-six and three percent faster than fiat-crypto. The microcode is, in short, the fastest 5×51 X25519 we measured; it is not the fastest X25519, remaining about eleven percent behind `amd64-64`, whose advantage is one of algorithm and native execution rather than of carries.

5.2 The Kernel: One Firing, Three Carry Chains

An entire field multiplication, or an entire field squaring, is performed by a single execution of one hooked macro-instruction — one firing. The native side issues that instruction; the patch then

computes every partial product, every inter-limb carry, and the final modular reduction within the register file, and returns with the five result limbs in place. Nothing about the operation loops inside the patch, and control never crosses back and forth between native code and microcode in the middle of an operation. The kernel is consequently a single flat schedule of triads, and its entire art lies in how densely the multiplications, accumulations, and carries are packed into the three operation slots of each triad.

Three accumulators that never share the carry flag. The kernel forms each output limb as a sum of products held in three independent running totals, none of which uses the architectural carry flag. The low halves of the partial products accumulate into a register we will call the low accumulator (TMP0); the high halves accumulate into a high accumulator (R8); and the carries thrown off by the low-half additions are summed into a third register (TMP9). The double-width product itself is split at the multiply: the instruction is encoded so that its low half is written into the operand register, where it is read and added into the low accumulator in the very next slot, while its high half is written into a scratch register (RCX) to be added into the high accumulator. The carry produced by each low-half addition is captured by a SETCC into a per-register flag domain of the kind established in Section 1.4, which is the property that makes the whole scheme possible: because each chain captures its carry into its own private flag, the carry capture fits in a spare slot of the same triad that issues the next multiplication, rather than serialising on one shared flag behind it. Listing 4 shows the resulting motif, in which a multiplication, the accumulation of a partial product, and a carry capture co-reside in two triads while the next product is already in flight.

Listing 4: The fused multiply–accumulate–carry motif, taken from the squaring kernel’s first output limb. TMP0 is the low accumulator, R8 the high accumulator, TMP9 the running sum of accumulation carries, TMP15 the transient SETCC carry. The multiply writes its low half into the operand register (read on the same line) and its high half into RCX (added in later).

{ ZEROEXT(TMP0,RAX),	MUL(RCX,RDI,RDI),	NOP,	NOP_SEQW }	// TMP0:=0 ; product a0*
<i>a0 -> (hi=RCX, lo=RDI)</i>				
{ ADD(TMP0,TMP0,RDI),	SETCC(TMP15,TMP0),	ADD(R8,R8,RCX),	NOP_SEQW }	// acc low half ;
<i>capture carry ; acc previous high half</i>				
{ ZEROEXT(TMP9,TMP15),	MUL(RCX,RBX,R13),	ADD(TMP0,TMP0,R13),	NOP_SEQW }	// seed carry sum ; next
<i>product ; acc its low half</i>				

Resolving a limb and linking to the next. At the end of a limb the three accumulators must yield two things: the finished value of this limb, and the carry that passes into the next. The sequence of Listing 5 produces both. The carry sum is first folded into the high accumulator, after which TMP0 holds the low part of the limb’s total and R8 the high part. The finished limb is just the portion of that total which fits within a limb; the carry is everything that protrudes above it.

Both fall out of one observation. A machine word is exactly one limb wide plus the thirteen spare bits that the representation leaves free (Section 5.1), so the carry arrives already in two pieces that cannot overlap. The piece taken from TMP0 is whatever spilled past the limb, and is no wider than those thirteen spare bits; the piece taken from R8 is shifted up by the same thirteen bits, so its lowest bits are empty. Because the two pieces sit in disjoint bit positions, joining them requires no addition — a single bitwise OR suffices, and that OR is the entire link between one limb and the next, seeding the low accumulator before the next limb’s products begin. The finished limb itself is isolated by a shift up followed by a shift back down, which discards the spare bits without a mask constant or a register to hold one.

Even after the last limb, one final carry is left over. It has run off the top of the field, so the reduction identity brings it back to the bottom: the kernel multiplies it by 19, adds it to the lowest limb, and passes one short carry into the next limb up. It then stops. The result is not fully tidied — one limb may end up a single bit too wide — but every limb has spare room, so the next operation reads it unchanged and the shortcut costs nothing.

Listing 5: Limb resolution and the inter-limb link. Because $2^{64} = 2^{51} \cdot 2^{13}$, the carry out of a limb splits into a 13-bit low part (TMP8) and a left-shifted high part (TMP1) whose bit ranges do not overlap, so the link that opens the next limb is an OR, not an addition.

```
{ ADD(R8,R8,TMP9),    SHL(TMP6,TMP0,13),    SHL(TMP1,R8,13),    NOP_SEQW } // fold carry sum ; mask-
    prep ; high carry = R8<<13
{ SHR(out0,TMP6,13),  SHR(TMP8,TMP0,51),    NOTAND(R8,R8,R8),  NOP_SEQW } // out0 = TMP0 mod 2^51 ;
    low carry ; clear R8
// ... the next limb opens with the link and its first product ...
{ OR(TMP0,TMP8,TMP1), MUL(RCX,R15,R13),    ADD(TMP0,TMP0,R13),NOP_SEQW } // TMP0 := lowcarry /
    highcarry ; next product
```

This is the field-arithmetic realisation of the structural claim the paper makes for the layer as a whole. The native multiplication is bound by a single carry chain threaded through one flag; the microcode runs three accumulation chains in parallel on private flags and folds each carry capture into a multiplication’s own triad. The work the layer performs — concurrent carry propagation — is exactly the work the instruction set cannot express, just as whole-state residency was for Keccak.

Authoring and verification. Unlike the generator-emitted Keccak round, these two kernels are scheduled by hand. At a few tens of triads each they are small enough to lay out manually, and their dense, irregular interleaving of multiplies and carries gives a generic packer little to improve; they are nonetheless held to the slot and hazard rules of Section 1.2. Each patch is checked against the fiat-crypto reference — the same backend the superoptimiser targets — over a large random corpus and a thousand-iteration self-chaining test, and the assembled scalar multiplication is checked against the RFC 7748 known-answer vectors before any timing is reported.

5.3 Installing Both Operations at Once

The multiplication and squaring patches reside in the patch RAM at the same time: the multiplication occupies 66 triads beginning at U7c00, and the squaring occupies 42 triads beginning immediately after it at U7d08. Since one hooked macro-instruction can be redirected to only a single patch entry point, holding two operations resident requires two match registers intercepting two different instructions, in the manner of Section 1.2. We hook the multiplication to `vmwrite` (opcode 0x0cd8) and the squaring to `vmread` (opcode 0x0618); the native side fires a multiplication by issuing a `vmwrite` and a squaring by issuing a `vmread`, the latter emitted as raw opcode bytes since the operation is otherwise inert in the unprivileged context. The combined $66 + 42 = 108$ triads sit within the 128-triad capacity with room to spare. This dual installation is not a convenience; it is what allows a single block of inline assembly to interleave squarings and multiplications freely, which is the foundation of the integration described in Section 5.5, since each kind of operation has its own dedicated entry instruction and the two never collide.

5.4 Triad Budget: Density, Not Capacity

For Keccak the binding constraint of Section 1.3 was capacity: a single round body strained against the 128-triad ceiling. For Curve25519 the constraint is inverted. At 108 triads the two operations

leave the budget unexhausted, and the design pressure is not whether the kernel fits but how tightly it packs, because every multiply–accumulate–carry sequence folded into a single triad shortens both the static patch and its dynamic critical path. The development of these kernels is largely a history of such merges. The multiplication began near 88 triads and was brought to 73 and then to 66 by, among other steps, packing the three preparatory reduction folds into one triad and pulling each product’s carry capture out of a dedicated triad and into the free third slot of a multiply–accumulate triad. The squaring’s last output limb was compressed from eight triads to seven by exploiting a cross-slot dependence within a single triad. Each such merge is legitimate only because the intra-triad hazard behaviour recorded in Section 1.4 guarantees that the three slots of a triad observe one another’s reads and writes in program order; the packing rests directly on that empirical result.

5.5 Assembling X25519 Around the Patches

A scalar multiplication invokes the field operations on the order of a thousand times: a 255-step Montgomery ladder with roughly ten field operations per step, followed by a Fermat inversion z^{p-2} that is itself a long chain of squarings. The fastest implementation we built, which we call the inlined ladder, expresses this entire computation as inline assembly built directly on the two patches, rather than as portable C that reaches them through function wrappers. The fifteen field elements a ladder step manipulates live together in one stack-resident state structure at fixed byte offsets, addressed throughout the step through a base register pinned to that structure, and the firings are emitted within one large assembly block per step. The compiler is thus obliged to allocate and spill only around the block as a whole, never between individual field operations.

Chaining through registers. The decisive integration technique is to choose the caller conventions so that one operation’s outputs land in exactly the registers the next operation reads as inputs, eliminating the store and reload that would otherwise separate them. The addition and subtraction macros are written to deposit their five result limbs in the same register set that the squaring and multiplication patches consume as their first operand. An addition that feeds a squaring, or a subtraction that feeds a multiplication, therefore becomes a single chain with no intervening memory traffic for the intermediate. The principal block of a ladder step contains six such chains; the simplest is the addition $A = x_2 + z_2$ feeding the squaring $AA = A^2$, and the others are $B = x_2 - z_2$ into B^2 , $D = x_3 - z_3$ into $D \cdot A$, $C = x_3 + z_3$ into $C \cdot B$, and the sum and difference of $D \cdot A$ and $C \cdot B$ each into a squaring. Two of these chains exist only because the four operations forming $D \cdot A$ and $C \cdot B$ are reordered away from their textbook sequence so that each difference or sum feeds its multiplication immediately, turning one chain into two without altering the dataflow. The three operations whose producer and consumer register sets do not coincide — the two final multiplications and the subtraction $E = AA - BB$ — are left to reload from the state structure, and a short tail block, together with one multiplication by the curve constant performed in C, completes z_2 as a seventh chain.

The inversion as one chained block. The Fermat inversion is built the same way, as a single inline block in which the 254 squarings are grouped into eleven runs of lengths 1, 2, 1, 5, 10, 20, 10, 50, 100, 50, 5, separated by eleven multiplications that recombine the intermediate powers. Within a run, only the first squaring loads its operand from memory and only the last stores its result; between consecutive squarings the previous output is renamed into the input position by four register moves, the fifth limb needing no move because the lowest output limb already occupies the lowest input register. The separating multiplications stand alone, because the register set a squaring produces does not match the set a multiplication consumes. This is the same data-movement principle as

Section 1.3, extended from the interior of one field operation, where the layer already removes the per-limb memory traffic, to the seams between operations, where the inlined ladder removes the reloads.

5.6 Performance

Methodology. The measurements follow the discipline of ???. The core is pinned to its base frequency so that the cycle counter and the core clock advance together, timings are taken within a single process, and each contender is swept over the twenty-four combinations of compiler and optimisation level, the best result for each being retained, in the manner of the SUPERCOP autotuner. Since the comparison is made at the level of a complete scalar multiplication, the aggregate figure for a backend is the geometric mean of its per-configuration ratios against the microcode.

The field operation in isolation. Timed on their own in a tight loop, the microcode field operations are faster than the fiat-crypto reference. The squaring completes in roughly 37 cycles, about 26% faster than fiat, and the multiplication in roughly 58 cycles, about 3% faster. The asymmetry is intrinsic to the kernels: squaring issues fifteen products against multiplication’s twenty-five, so it has proportionally more slack into which the carry work can be packed, whereas the multiplication is closer to multiplier-throughput bound and the layer’s carry-concurrency advantage has less room to act. Because a ladder and a Fermat inversion are both squaring-dominated, the larger squaring advantage is what propagates into the end-to-end result.

End to end. Table 4 reports the best per-contender result across the sweep, in median cycles per scalar multiplication. The inlined microcode ladder completes in about 300,000 cycles and is the fastest of every 5×51 contender we measured: against the field backends it is a geometric mean of roughly $1.25\times$ faster than fiat-crypto, $1.30\times$ faster than hand-written `__uint128_t` C, $1.28\times$ faster than the CryptOpt superoptimiser, and $1.31\times$ faster than `donna_c64`, and it is also faster than the hand-written `amd64-51` assembly stack. The field-operation advantage is not an artefact of our own ladder: when the microcode field operations are carried into the Bernstein–Schwabe `amd64-51` framework, that hybrid is a geometric mean of about $1.08\times$ faster than the same framework running its authors’ own assembly field operations, so the advantage survives transplantation into an independently engineered scalar multiplication.

Where the advantage ends. The fastest scalar multiplication on this core is nonetheless not ours. It is the Bernstein–Schwabe `amd64-64` implementation, which uses the saturated 4×64 representation in radix 2^{64} and completes in 271,326 cycles, roughly 11% ahead of the inlined microcode ladder at about 300,000. More revealing for the layer is what happens when we encode the field operations themselves in that 4×64 representation: the microcode port runs in 511,017 cycles, a geometric mean of about $1.77\times$ slower than our 5×51 microcode and nearly twice the cost of the native 4×64 assembly. The cause is exactly the carry structure of Section 5.1. A saturated 4×64 multiplication produces long, full-width carry chains across 64-bit limbs, and the add-with-carry instruction those chains want reads the architectural flag domain that, as recorded in Section 1.4, is frozen on entry to a patch and cannot be threaded through a sequence of dependent additions inside the microcode. The four-deep saturated carry must therefore be emulated by serial SETCC operations, which reintroduces precisely the serialisation that the 5×51 kernel’s three parallel chains were built to avoid. The representation that suits the native core best is thus the one that

Table 4: End-to-end X25519, best configuration per contender over the twenty-four-configuration sweep, in median cycles. The inlined microcode ladder is the fastest 5×51 implementation but is overtaken by the hand-written 4×64 saturated assembly, and the microcode port to that same 4×64 representation is the slowest entry by a wide margin.

contender	representation and backend	median (cyc)
amd64-64 / asm	4×64 saturated, hand asm	271 326
ours / ucode-inline	5×51 , microcode (inlined ladder)	299 967
amd64-51 / ucode	5×51 , microcode (B-S framework)	314 416
donna_c64	5×51 , portable C	335 744
ours / fiat	5×51 , fiat-crypto C	354 851
amd64-51 / asm	5×51 , hand asm	356 038
ours / hand-C	5×51 , hand-written C	379 433
ours / cryptopt	5×51 , CryptOpt asm	380 361
amd64-64 / ucode	4×64 saturated, microcode	511 017

suits the layer worst, and the representation at which the layer excels does not yield the fastest algorithm overall.

Summary. At the level at which the paper stakes its claim — the field operation, with the ladder, inversion, and conditional swap held in common — the microcode is the fastest Curve25519 field backend we measured on this core, ahead of an optimising compiler, a formally verified generator, a cryptographic superoptimiser, and expert hand assembly, and the advantage carries through both our own scalar multiplication and the Bernstein–Schwabe framework. The result has the same shape as the Keccak result of ??: it comes from a structural per-operation capability the instruction set cannot reach, here the fusion of multiply, accumulate, and carry capture into single triads together with three concurrent SETCC carry chains. And as with Keccak the limit is stated without softening. The fastest X25519 on this core belongs to a different number representation, the saturated 4×64 form, and that representation is exactly where the one structural weakness of the layer — long carry chains under a frozen single-flag model — is laid bare.